

Análisis de Algoritmos

DFS recursivo y árboles de recorrido; componentes conexas

Luis Alfredo Alvarado Rodríguez

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
ESCUELA DE CIENCIAS FÍSICAS Y MATEMÁTICAS

9 de octubre de 2025

- 1 DFS recursivo y árboles de recorrido
 - Motivación y contexto
 - Definición y algoritmo
 - Árboles de recorrido
- 2 Componentes conexas
 - Definición y algoritmo
- 3 Resumen/Conclusiones

Motivación: recursión en grafos

Idea general

- Muchos problemas en grafos admiten **planteamientos recursivos**.

Motivación: recursión en grafos

Idea general

- Muchos problemas en grafos admiten **planteamientos recursivos**.
- Pero el coste puede ser **exponencial** (p. ej., conjunto independiente máximo).

Motivación: recursión en grafos

Idea general

- Muchos problemas en grafos admiten **planteamientos recursivos**.
- Pero el coste puede ser **exponencial** (p. ej., conjunto independiente máximo).
- Patrón: reducir G a grafos más pequeños y combinar soluciones.

Motivación: recursión en grafos

Idea general

- Muchos problemas en grafos admiten **planteamientos recursivos**.
- Pero el coste puede ser **exponencial** (p. ej., conjunto independiente máximo).
- Patrón: reducir G a grafos más pequeños y combinar soluciones.

Ejemplo: maxset

Estamos buscando el tamaño del mayor conjunto independiente de vértices de G :

$$\text{maxset}(G) = \max\left(\text{maxset}(G - \{v^*\}), 1 + \text{maxset}(G - \{v^*\} - \text{Nbhd}(v^*))\right).$$

Complejidad de un esquema recursivo

Desigualdad recurrente para `maxset1`

$$f(n) \leq c n^2 + f(n-1) + f(n-2) \quad (2.13)$$

Complejidad de un esquema recursivo

Desigualdad recurrente para `maxset1`

$$f(n) \leq c n^2 + f(n-1) + f(n-2) \quad (2.13)$$

Por el Teorema 1.4.1, $f(n) = O((1,619)^n)$.

Complejidad de un esquema recursivo

Desigualdad recurrente para `maxset1`

$$f(n) \leq cn^2 + f(n-1) + f(n-2) \quad (2.13)$$

Por el Teorema 1.4.1, $f(n) = O((1,619)^n)$.

Mejora estructural

Al forzar v^* con $\deg(v^*) \geq 2$ (algoritmo `MAXSET2`):

$$f(n) \leq cn^2 + f(n-1) + f(n-3) \quad (2.14)$$

y se obtiene $O(1,47^n)$ (raíz positiva de $c^3 = c^2 + 1$).

Moraleja

No toda recursión es “barata”. DFS, en cambio, será **lineal** en el tamaño del grafo.

¿Qué es DFS?

Definición 1

*La **búsqueda en profundidad** (DFS) es un procedimiento recursivo que, dado un grafo $G = (V, E)$ y un vértice origen s , explora sistemáticamente los vértices alcanzables desde s siguiendo recursivamente aristas hacia vértices no visitados hasta “profundizar” lo más posible antes de retroceder.*

¿Qué es DFS?

Definición 1

*La **búsqueda en profundidad** (DFS) es un procedimiento recursivo que, dado un grafo $G = (V, E)$ y un vértice origen s , explora sistemáticamente los vértices alcanzables desde s siguiendo recursivamente aristas hacia vértices no visitados hasta “profundizar” lo más posible antes de retroceder.*

- Entrada: G (listas de adyacencia preferidas).

¿Qué es DFS?

Definición 1

*La **búsqueda en profundidad** (DFS) es un procedimiento recursivo que, dado un grafo $G = (V, E)$ y un vértice origen s , explora sistemáticamente los vértices alcanzables desde s siguiendo recursivamente aristas hacia vértices no visitados hasta “profundizar” lo más posible antes de retroceder.*

- Entrada: G (listas de adyacencia preferidas).
- Estado: arreglo `visitado[v]` y, opcionalmente, tiempos de descubrimiento/finalización.

¿Qué es DFS?

Definición 1

*La **búsqueda en profundidad** (DFS) es un procedimiento recursivo que, dado un grafo $G = (V, E)$ y un vértice origen s , explora sistemáticamente los vértices alcanzables desde s siguiendo recursivamente aristas hacia vértices no visitados hasta “profundizar” lo más posible antes de retroceder.*

- Entrada: G (listas de adyacencia preferidas).
- Estado: arreglo `visitado[v]` y, opcionalmente, tiempos de descubrimiento/finalización.
- Salida: **árbol(es) de recorrido** y orden de visita.

DFS recursivo (pseudocódigo) y coste

Pseudocódigo

1. Para cada $v \in V$: `visitado[v] := falso`.

DFS recursivo (pseudocódigo) y coste

Pseudocódigo

1. Para cada $v \in V$: `visitado[v] := falso`.
2. Para cada $u \in V$:
 - 1) Si `visitado[u]` es falso, llamar `DFS(u)`.

DFS recursivo (pseudocódigo) y coste

Pseudocódigo

1. Para cada $v \in V$: `visitado[v] := falso`.
2. Para cada $u \in V$:
 - 1) Si `visitado[u]` es falso, llamar `DFS(u)`.
3. `DFS(u)`:
 - 1) Marcar `visitado[u] := verdadero`.
 - 2) Para cada $w \in \text{Adj}(u)$:
 - 1) Si `visitado[w]` es falso, registrar arista (u, w) como *de árbol* y llamar `DFS(w)`.

DFS recursivo (pseudocódigo) y coste

Pseudocódigo

1. Para cada $v \in V$: $\text{visitado}[v] := \text{falso}$.
2. Para cada $u \in V$:
 - 1) Si $\text{visitado}[u]$ es falso, llamar $\text{DFS}(u)$.
3. $\text{DFS}(u)$:
 - 1) Marcar $\text{visitado}[u] := \text{verdadero}$.
 - 2) Para cada $w \in \text{Adj}(u)$:
 - 1) Si $\text{visitado}[w]$ es falso, registrar arista (u, w) como *de árbol* y llamar $\text{DFS}(w)$.

Complejidad

Con listas de adyacencia: $\Theta(|V| + |E|)$ en tiempo y $O(|V|)$ en pila (profundidad $\leq |V|$).

Árbol(es) de recorrido de DFS

Definición 2

*El **árbol de DFS** desde s tiene como aristas las primeras que conducen a cada vértice no visitado durante $DFS(s)$. Ejecutando DFS desde cada vértice no alcanzado se obtiene un **bosque DFS**.*

Árbol(es) de recorrido de DFS

Definición 2

*El **árbol de DFS** desde s tiene como aristas las primeras que conducen a cada vértice no visitado durante $DFS(s)$. Ejecutando DFS desde cada vértice no alcanzado se obtiene un **bosque DFS**.*

- En grafos no dirigidos: toda arista es *de árbol* o *de retroceso* (conecta con un ancestro).

Árbol(es) de recorrido de DFS

Definición 2

*El **árbol de DFS** desde s tiene como aristas las primeras que conducen a cada vértice no visitado durante $DFS(s)$. Ejecutando DFS desde cada vértice no alcanzado se obtiene un **bosque DFS**.*

- En grafos no dirigidos: toda arista es *de árbol* o *de retroceso* (conecta con un ancestro).
- Cada componente conexa induce exactamente un árbol en el bosque DFS.

Árbol(es) de recorrido de DFS

Definición 2

*El **árbol de DFS** desde s tiene como aristas las primeras que conducen a cada vértice no visitado durante $DFS(s)$. Ejecutando DFS desde cada vértice no alcanzado se obtiene un **bosque DFS**.*

- En grafos no dirigidos: toda arista es *de árbol* o *de retroceso* (conecta con un ancestro).
- Cada componente conexa induce exactamente un árbol en el bosque DFS.
- En una componente con n_c vértices, el árbol DFS tiene $n_c - 1$ aristas.

Definición 3

Una **componente conexa** de un grafo no dirigido G es un subgrafo inducido maximal en el que todo par de vértices está conectado por algún camino.

Definición 3

Una **componente conexas** de un grafo no dirigido G es un subgrafo inducido maximal en el que todo par de vértices está conectado por algún camino.

Idea con DFS

- Lanzar $\text{DFS}(u)$ sobre cada vértice no visitado.

Definición 3

Una **componente conexas** de un grafo no dirigido G es un subgrafo inducido maximal en el que todo par de vértices está conectado por algún camino.

Idea con DFS

- Lanzar $\text{DFS}(u)$ sobre cada vértice no visitado.
- Todos los vértices alcanzados desde u forman **una** componente.

Definición 3

Una **componente conexas** de un grafo no dirigido G es un subgrafo inducido maximal en el que todo par de vértices está conectado por algún camino.

Idea con DFS

- Lanzar $\text{DFS}(u)$ sobre cada vértice no visitado.
- Todos los vértices alcanzados desde u forman **una** componente.
- Repetir hasta visitar todos los vértices.

Algoritmo para componentes (DFS)

Pseudocódigo

1. `comp := 0.`

Algoritmo para componentes (DFS)

Pseudocódigo

1. `comp := 0`.
2. Para cada $u \in V$:
 - 1) Si `visitado[u]` es falso:
 - 1) `comp := comp + 1`; etiquetar u con `comp`.
 - 2) `DFS_marcar(u, comp)` que propaga la etiqueta a todo vértice alcanzable.

Algoritmo para componentes (DFS)

Pseudocódigo

1. `comp := 0`.
2. Para cada $u \in V$:
 - 1) Si `visitado[u]` es falso:
 - 1) `comp := comp + 1`; etiquetar u con `comp`.
 - 2) `DFS_marcar(u, comp)` que propaga la etiqueta a todo vértice alcanzable.
3. Salida: etiquetas de componentes y el bosque DFS.

Algoritmo para componentes (DFS)

Pseudocódigo

1. $\text{comp} := 0$.
2. Para cada $u \in V$:
 - 1) Si $\text{visitado}[u]$ es falso:
 - 1) $\text{comp} := \text{comp} + 1$; etiquetar u con comp .
 - 2) $\text{DFS_marcar}(u, \text{comp})$ que propaga la etiqueta a todo vértice alcanzable.
3. Salida: etiquetas de componentes y el bosque DFS.

Coste

Tiempo $\Theta(|V| + |E|)$, espacio $O(|V|)$. Cada arista se examina a lo sumo dos veces (no dirigido).

Ejemplo (sin dibujo)

Grafo G

$V = \{1, 2, 3, 4, 5, 6\}; \quad E = \{(1, 2), (2, 3), (4, 5)\}.$

Ejemplo (sin dibujo)

Grafo G

$V = \{1, 2, 3, 4, 5, 6\}; \quad E = \{(1, 2), (2, 3), (4, 5)\}.$

Ejecución

- Iniciar en 1: DFS visita $\{1, 2, 3\} \Rightarrow$ componente $C_1 = \{1, 2, 3\}.$

Ejemplo (sin dibujo)

Grafo G

$V = \{1, 2, 3, 4, 5, 6\}; \quad E = \{(1, 2), (2, 3), (4, 5)\}.$

Ejecución

- Iniciar en 1: DFS visita $\{1, 2, 3\} \Rightarrow$ componente $C_1 = \{1, 2, 3\}.$
- Siguiendo no visitado: 4 $\Rightarrow$ DFS visita $\{4, 5\} \Rightarrow C_2 = \{4, 5\}.$

Ejemplo (sin dibujo)

Grafo G

$V = \{1, 2, 3, 4, 5, 6\}; \quad E = \{(1, 2), (2, 3), (4, 5)\}.$

Ejecución

- Iniciar en 1: DFS visita $\{1, 2, 3\} \Rightarrow$ componente $C_1 = \{1, 2, 3\}.$
- Siguiente no visitado: 4 $\Rightarrow$ DFS visita $\{4, 5\} \Rightarrow C_2 = \{4, 5\}.$
- Siguiente no visitado: 6 $\Rightarrow C_3 = \{6\}.$

Ejemplo (sin dibujo)

Grafo G

$V = \{1, 2, 3, 4, 5, 6\}; \quad E = \{(1, 2), (2, 3), (4, 5)\}.$

Ejecución

- Iniciar en 1: DFS visita $\{1, 2, 3\} \Rightarrow$ componente $C_1 = \{1, 2, 3\}.$
- Siguiente no visitado: 4 $\Rightarrow$ DFS visita $\{4, 5\} \Rightarrow C_2 = \{4, 5\}.$
- Siguiente no visitado: 6 $\Rightarrow C_3 = \{6\}.$
- Resultado: 3 componentes; bosque DFS con tres árboles.

- DFS es un **esquema recursivo lineal** para explorar grafos.

- DFS es un **esquema recursivo lineal** para explorar grafos.
- Su bosque de recorrido estructura G y **segmenta** en componentes conexas.

- DFS es un **esquema recursivo lineal** para explorar grafos.
- Su bosque de recorrido estructura G y **segmenta** en componentes conexas.
- Algoritmo de componentes: repetir DFS en no visitados; coste $\Theta(|V| + |E|)$.

- DFS es un **esquema recursivo lineal** para explorar grafos.
- Su bosque de recorrido estructura G y **segmenta** en componentes conexas.
- Algoritmo de componentes: repetir DFS en no visitados; coste $\Theta(|V| + |E|)$.
- El libro ilustra otros esquemas recursivos (maxset, polinomio cromático) con **coste exponencial**.

- DFS es un **esquema recursivo lineal** para explorar grafos.
- Su bosque de recorrido estructura G y **segmenta** en componentes conexas.
- Algoritmo de componentes: repetir DFS en no visitados; coste $\Theta(|V| + |E|)$.
- El libro ilustra otros esquemas recursivos (maxset, polinomio cromático) con **coste exponencial**.
- Lección: la recursión es una **técnica**, no una garantía de eficiencia; la elección del subproblema importa.